Ciurea Ana Sorina 331CC

Tema 2 APD - Protocolul CHORD in MPI

Acest proiect implementeaza o simulare a protocolului **CHORD** (Distributed Hash Table) folosind **MPI** (Message Passing Interface). Fiecare proces MPI reprezinta un nod in inelul CHORD care comunica cu celelalte noduri pentru a rezolva cereri de lookup.

Arhitectura si Implementare

Protocolul este implementat intr-un mediu static, dar respecta $O(\log N)$ folosind Finger Tables

1. Construirea Finger Table (`build_finger_table`)

Deoarece inelul este static, tabela finger este construita la initializare:

1.1. Pentru fiecare intrare i (de la 0 la $M-1$), se calculeaza pozitia de start: `start = (id + 2^i) % RING_SIZE`.

1.2. Se cauta succesorul acestei pozitii in lista sortata a tuturor nodurilor existente (`sorted_ids`).

1.3. Functia `find_successor_simple` parcurge lista nodurilor pentru a gasi primul nod cu ID $\geq$ start.

2. Rutarea Eficienta (`closest_preceding_finger`)

Aceasta functie este esentiala pentru performanta logaritmica. Cand un nod trebuie sa trimita o cerere mai departe, cauta in tabela sa finger cel mai potrivit nod:

2.1. Se itereaza prin finger table de la $M-1$ la 0.

2.2. Se verifica daca `finger[i].node` se afla in intervalul circular `(self.id, key)`.

2.3. Primul nod gasit care respecta conditia este returnat ca "urmatorul hop".

2.4. Daca nu se gaseste niciun finger potrivit, se returneaza succesorul direct (`self.successor`) ca fallback.

3. Gestionarea Cererilor (`handle_lookup_request`)

Functia proceseaza mesajele de tip `TAG_LOOKUP_REQ`:

3.1. Adauga ID-ul curent in istoricul mesajului.

3.2. Verifica daca nodul curent sau succesorul sau este responsabil pentru cheie:
* Daca `key` $\in$ `(self.id, self.successor]`, atunci succesorul este responsabil.

3.3. Daca s-a gasit responsabilul, se trimite `TAG_LOOKUP_REP` inapoi la initiator.

3.4. Altfel, se foloseste `closest_preceding_finger` pentru a determina urmatorul nod si se trimite cererea mai departe (`TAG_LOOKUP_REQ`).

4. Protocolul de Terminare

Sistemul distribuite functioneaza pana cand toate cererile au fost rezolvate:

4.1. Fiecare nod contorizeaza cate cereri a initiat si cate raspunsuri (`TAG_LOOKUP_REP`) a primit.

4.2. Cand un nod isi termina toate cererile locale, trimite un mesaj de difuzare `TAG_DONE` catre toate celelalte noduri.

4.3. Nodul se opreste doar atunci cand a terminat propriile cereri si a primit `TAG_DONE` de la toti ceilalti participanti.

Punctaj obtinut

Corectitudine: 40/40

Eficienta: 40/40

Total: 80/80